

Autonomous Vision-Based Landing of a Parrot Mambo Drone on a Moving Platform

Chinmay Amrutkar¹

¹Arizona State University, Tempe, AZ

Keywords:

Parrot Mambo
Drone
MATLAB
Color Detection
Color Threshold
Moving Platform
Autonomous Landing
Vision-Based Navigation
UAV

ABSTRACT

The integration of aerial and ground robotics has significant potential for developing intelligent multi-agent systems capable of collaborative tasks. This project presents a vision-based system enabling a Parrot Mambo minidrone to autonomously detect and land on a moving line-follower robot. Using HSV color segmentation and blob analysis, the drone identifies a colored platform beneath it and computes its position error in real-time. These error signals guide a path planning module that transitions the drone through hover, cruise, and descent phases. During the cruise, velocity commands are adjusted based on x and y errors, while altitude is gradually reduced for a smooth landing. The system ensures precise timing, allowing the drone to land either before or at the end of the line. Developed entirely in MATLAB and Simulink, this approach demonstrates real-time aerial-ground coordination using onboard visual feedback and modular control logic.

Corresponding Author:

Chinmay Ravindra Amrutkar
Arizona State University, Tempe, AZ, USA
Email: chinmay.amrutkar@asu.edu

1. INTRODUCTION

The integration of vision-based control in unmanned aerial vehicles (UAVs) has enabled a range of autonomous applications, including navigation, surveillance, and precision landing. In particular, the Parrot Mambo minidrone—equipped with an onboard downward-facing camera and supported by Simulink’s model-based development tools—offers a flexible platform for rapid prototyping of such autonomous systems. This project addresses the challenge of enabling the Parrot Mambo drone to detect, approach, and land on a **moving ground vehicle**, specifically a line-follower robot, using **real-time visual feedback**. The system must identify a uniquely colored platform affixed to the mobile base and initiate a **controlled descent only when the drone is properly aligned** above it. Furthermore, the descent must be **timed precisely**, such that the drone can land either during the motion or at the terminal point of the line, depending on platform velocity and alignment.

The core solution involves two primary subsystems:

1. An **image processing module**, which uses HSV-based color segmentation and blob analysis to detect the platform and estimate its centroid position.
2. A **path planning and control module**, which processes the positional error (x, y) to generate velocity commands, enabling the drone to align with the moving target and gradually reduce altitude during approach.

By integrating these components within MATLAB and Simulink, the system demonstrates how **vision-driven control and finite-state motion planning** can be used to achieve real-time coordination between aerial and ground-based robots.

2. SETTING UP THE PARROT MAMBO DRONE

To successfully implement real-time autonomous navigation with the Parrot Mambo drone, it is essential to first establish a stable connection between the drone and the development environment. This setup involves installing the necessary drivers, configuring the communication interface, and verifying hardware functionality through initial deployment tests.

1. Installing Bluetooth Drivers and Establishing Connection:

The Parrot Mambo drone connects to MATLAB and Simulink via Bluetooth, requiring proper driver installation on a Windows system. Following the guidelines from MathWorks, the appropriate Bluetooth drivers were installed to ensure stable wireless communication. Once the drivers were set up, the connection was verified by pairing the drone with the computer and confirming its availability in the Simulink support package.

2. Deploying the Parrot Getting Started Model:

To validate the connection and test the drone's hardware, the `parrot_gettingstarted` model from the MathWorks documentation was deployed. This model executes a basic motor test by spinning opposite motors alternately, ensuring that all four motors function correctly. Successful execution of this test confirmed that the drone was properly configured and ready for further experimentation.

By completing this setup, the foundation was established for integrating the drone with MATLAB-based image processing and control systems, enabling further development in color-based object detection and UAV automation.



Figure 1. Parrot Mambo Drone

3. METHODOLOGY

Following the successful setup of the Parrot Mambo drone, this experiment was conducted in a structured manner to achieve autonomous navigation integrated with real-time color detection and autonomous landing. The core of this project lies in the Simulink model design, integrating real-time image processing with Stateflow-based control logic to achieve autonomous line following and landing.

3.1. Simulink Model Setup and Framework Analysis

- Launch and Template Selection:
 - Begin by launching Simulink and open *parrotMinidroneCompetitionStart* template, by entering *parrotMinidroneCompetitionStart* in Command Window. This template establishes the basic framework required for integrating image processing with flight control.

- Framework Analysis:
 - Carefully analyze the pre-configured image processing block to understand its workflow and ensure that it can interface seamlessly with the drone's downward-facing camera.

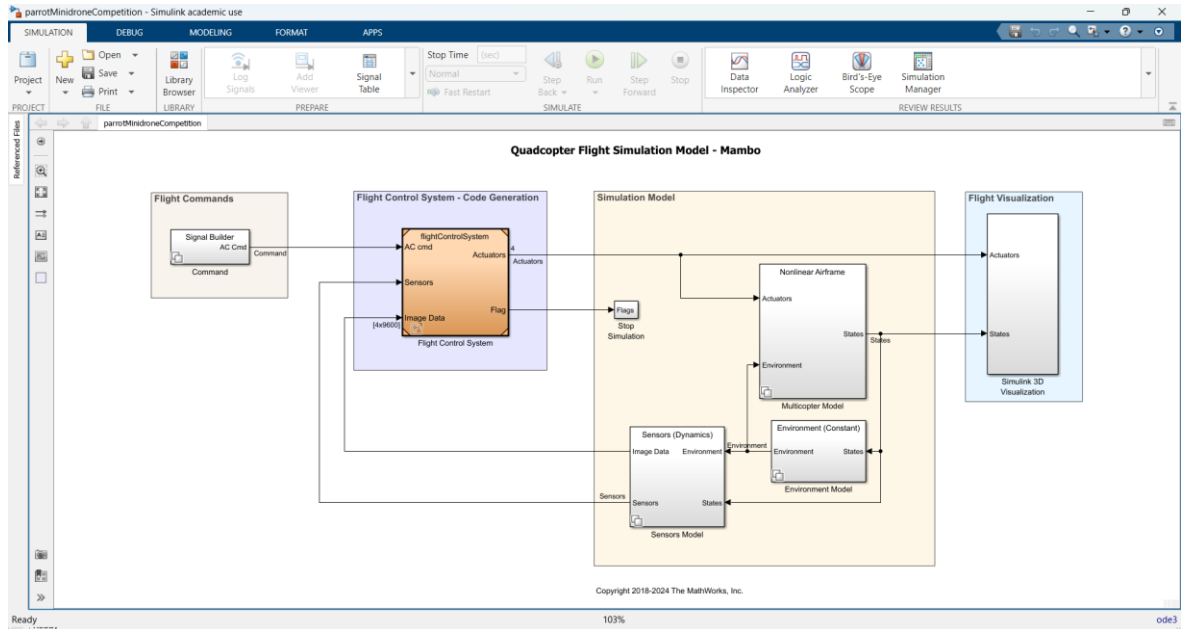


Figure 2. Quadcopter Flight Simulation Model – Mambo project

3.2. Image processing System

The image processing system is the visual backbone of the drone's autonomous descent framework. It enables the Parrot Mambo minidrone to detect a **colored platform placed on a mobile line-follower robot** and calculate its relative position in real time. This is achieved using **HSV-based color segmentation**, followed by **blob analysis** to extract the **centroid** of the detected area, which is used to generate error signals for path correction and landing alignment. The pipeline begins with the **PARROT Image Conversion** block, which converts the Y1UY2V input stream from the drone's camera to standard **RGB format**. This RGB image is then converted to **HSV color space** using the function shown in Figure 5, which was generated using MATLAB's Color Thresholding App Figure 4, where a red-colored platform shown in Figure 3 was sampled and threshold bounds were carefully selected to isolate only the pixels corresponding to the line. The result is a binary mask: a black and white image where white pixels (value 1) indicate the presence of the desired color (red), and black pixels (value 0) represent the background.

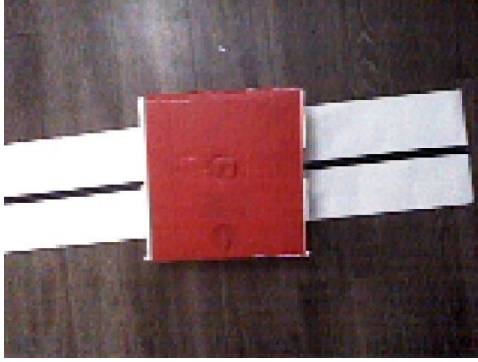


Fig. 3. Captured Image

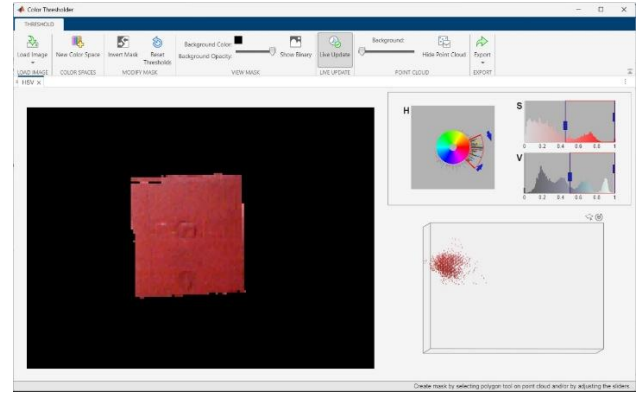


Fig. 4. Color Thresholding App (fine tuned for red color)

```

red_color
function BW = red_color(R,G,B)

% HSV
I = cat(3,R,G,B);

I = rgb2hsv(I);

% Define thresholds for channel 1 based on histogram settings
channel1Min = 0.832;
channel1Max = 0.057;

% Define thresholds for channel 2 based on histogram settings
channel2Min = 0.347;
channel2Max = 1.000;

% Define thresholds for channel 3 based on histogram settings
channel3Min = 0.483;
channel3Max = 1.000;

sliderBW = ( (I(:,:,1) >= channel1Min) | (I(:,:,1) <= channel1Max) ) & ...
(I(:,:,2) >= channel2Min) & (I(:,:,2) <= channel2Max) & ...
(I(:,:,3) >= channel3Min) & (I(:,:,3) <= channel3Max);
BW = sliderBW;

end

```

Fig. 5. Code inside the function block auto-generated code from colorThresholder app for red color

To ensure robustness in the presence of small-scale noise or inconsistencies in color detection, the binary image is passed through a **Median Filter**. This filter smoothens the mask by removing salt-and-pepper noise while preserving the larger features of the segmented line. After this, the processed binary image is routed to a **Video Viewer** block, which displays the detection results in real time during both simulations and live flight. Another parallel Video Viewer is connected to the original RGB stream, allowing the operator to visually verify the accuracy of the segmentation.

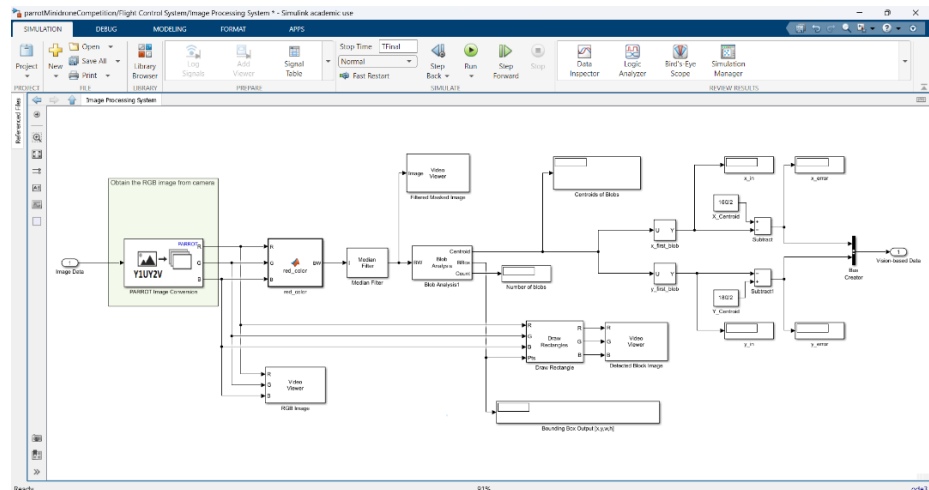


Fig. 6. Image Processing Block Subsystems

The filtered binary mask is then fed to the **Blob Analysis block**, which identifies the largest contiguous region (blob) corresponding to the platform. The block outputs the **centroid coordinates (x, y)**, the **bounding box**, and the **number of blobs detected**. The largest blob is used for position estimation, and its centroid indicates the platform's position in the image frame. These centroid values are then compared to the image center (e.g., 160/2 in width and 120/2 in height). The differences yield the **x_error** and **y_error**, which represent how far the drone is from being directly above the center of the platform in both directions. These error signals are critical for path correction and are sent forward to the path planning block.

Additionally, a bounding box is drawn around the detected blob and displayed along with the RGB and binary mask images using **Video Viewer** blocks, allowing for live visual feedback during testing and flight. Figure 6 illustrates the complete Simulink Image Processing Block, highlighting the full flow: from camera input to HSV mask generation, blob detection, and centroid-based error calculation.

3.3. Stateflow Control System

The path planning module functions as the decision-making and trajectory control core for the autonomous landing system. Its purpose is to compute real-time position commands for the Parrot Mambo drone based on visual feedback from the image processing block and state estimation data from the onboard sensors. Implemented entirely within Simulink using **Stateflow**, this subsystem enables dynamic control over horizontal motion and altitude during the landing approach, aligning the drone with the moving line-follower platform.

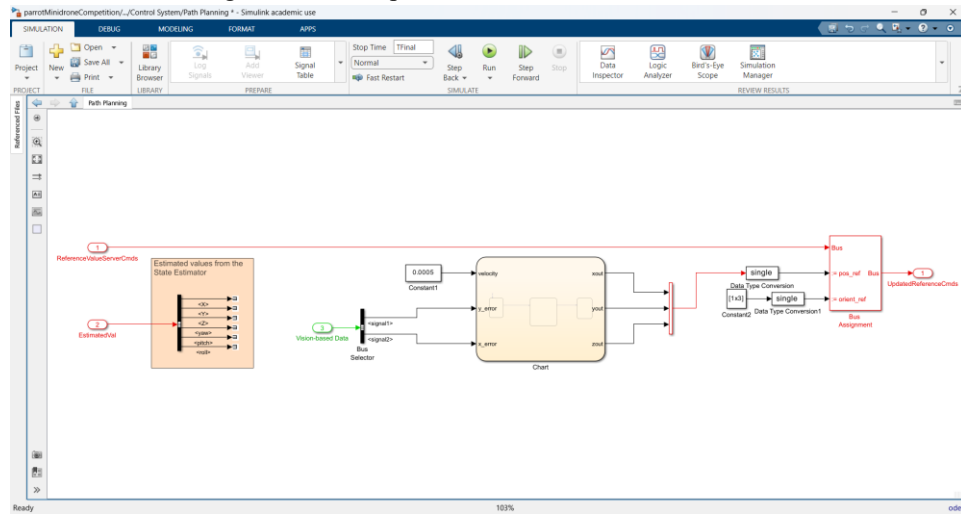


Fig. 7. Simulink implementation of the path planning module, integrating estimated drone state and visual error signals to compute real-time position references.

As shown in Figure 7, the path planning block receives two primary inputs: the **Vision-based Data** bus and the **EstimatedVal** signal from the onboard state estimator. The vision data consists of the x and y centroid errors computed from the detected platform position in the image frame while the state estimator provides the drone's real-time position and orientation (x, y, z, yaw, pitch, roll). A constant velocity input (e.g., 0.0005) is also fed to the control logic to scale the response magnitude.

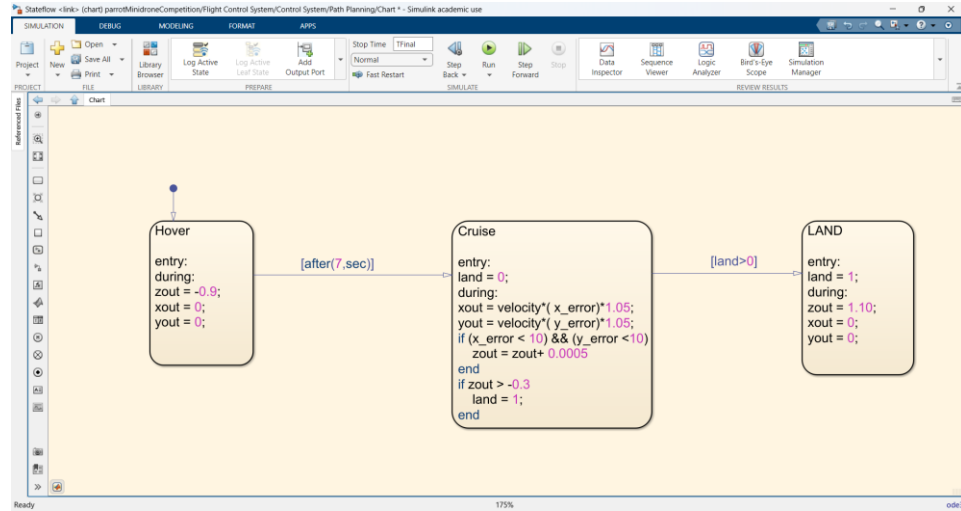


Fig. 8. Stateflow chart detailing the drone's behavioral transitions from hover to cruise and landing, based on timing, visual alignment, and error thresholds.

At the heart of the path planning module lies the **Stateflow chart** shown in Figure 8, which governs the drone's state transitions between hovering, cruising, and landing. Upon takeoff, the drone enters the **Hover** state, where it holds a constant altitude ($zout = -0.9$) and maintains a fixed horizontal position ($xout = yout = 0$). This initial phase lasts for a defined duration of 7 seconds (`[after(7, sec)]`), allowing the drone to stabilize before executing trajectory control. Once the hover duration elapses, the system transitions to the **Cruise** state, which forms the operational core of the descent logic. In this state, the controller uses the visual centroid errors x_error and y_error to compute new position references using proportional control laws. These references are scaled using the defined velocity gain and correction factor (e.g., $xout = velocity * x_error * 1.05$). This ensures that the drone incrementally corrects its position as it tracks the mobile platform beneath. The updated $xout$ and $yout$ values are then sent to the actuators via the `pos_ref` output signal. A particularly critical feature of this state is its **gradual descent mechanism**, which is conditionally activated based on spatial alignment. When both x_error and y_error fall below a predefined threshold (indicating the drone is nearly centered above the platform), the vertical position reference $zout$ is incrementally increased by `0.0005` in each control cycle, thereby initiating a slow and smooth descent. This conditional descent strategy prevents premature landing and ensures the drone only descends when directly over the moving platform. The descent continues until $zout$ exceeds a threshold of `-0.3`, upon which a flag (`land = 1`) is raised, triggering the final state transition.

The **Land** state is designed to command a vertical-only descent with complete suppression of horizontal motion. The drone's x and y references are frozen ($xout = yout = 0$), and the $zout$ is assigned a high positive value (`zout = 1.10`) to execute a stable and deliberate touchdown. This separation of motion responsibilities horizontal alignment in cruise followed by vertical descent in landing ensures a high degree of precision and safety. The output of this chart, encapsulated in the `UpdatedReferenceCmds` bus, is routed to the drone's flight control system. It contains both position and orientation references (`pos_ref` and `orient_ref`), updated in real-time based on vision feedback and internal logic. The structure and modularity of this approach make it robust, interpretable, and suitable for real-time onboard execution. The clear delineation between control phases (hover, cruise, land) enables seamless integration of vision-based cues with low-level motion primitives, resulting in a capable autonomous landing system that adapts to a dynamic ground target.

3.4. Deployment of the Integrated System on the Parrot Mambo Drone

After successful simulation and validation of the image processing and path planning modules, the fully integrated system was deployed onto the Parrot Mambo minidrone using the Simulink hardware support package. This final stage translated the abstract control architecture into a practical, embedded

solution that executed autonomously on the drone's onboard processor. Deployment began with configuration of Simulink's hardware settings to target the Parrot Mambo board. All logic components—ranging from HSV color segmentation and blob-based centroid tracking to path planning and Stateflow-based flight control—were compiled and deployed as a unified code package. The **Build, Deploy & Start** function allowed seamless transfer of the model and real-time execution without the need for an external host system during flight.

Once deployed, the drone executed the mission profile autonomously. Upon power-up, it ascended into a stable **hover** state, holding a fixed altitude for stabilization. After a predefined duration, it transitioned into the **cruise** state, where it began processing live video data from its downward-facing camera. The HSV-based segmentation module reliably isolated the colored landing platform, and the centroid of the detected blob was extracted and used to guide the drone's lateral alignment over the target. As alignment improved and the positional errors reduced below a set threshold, the descent logic embedded within the Stateflow controller began incrementally lowering the drone's altitude. This **controlled descent** continued until the landing conditions were satisfied, upon which the system transitioned into a dedicated **Land** state and executed a stable vertical descent onto the target platform.



Fig. 10. Test environment showing the Parrot Mambo drone approaching a moving red platform mounted on a line-follower robot during autonomous landing trials.

To validate the system's behavior in a realistic setting, the drone was tested in an environment where the colored landing platform was affixed to a mobile **line-follower robot**, as shown in Figure 10. The robot followed a black line over a light-colored floor, creating a dynamic landing target. The drone successfully tracked the platform in motion, dynamically correcting its position and initiating descent either mid-path or at the line's end, depending on timing and alignment. This experimental setup confirmed the robustness of the real-time embedded implementation. The drone responded consistently to live visual data, executed smooth transitions between control states, and achieved precise touchdown, thereby demonstrating the effectiveness of the integrated aerial-ground coordination system.

4. RESULTS AND DISCUSSION

The complete integrated system—consisting of real-time HSV-based image processing, centroid error tracking, and Stateflow-based flight control—was successfully tested on the Parrot Mambo drone in a real-world environment featuring a mobile landing platform. The experiments confirmed the system's ability to achieve visually guided precision landing on a moving base. During multiple trial runs, the drone consistently maintained stable flight, demonstrating precise lateral correction while tracking the colored platform in motion. The HSV thresholding mechanism proved effective under controlled lighting conditions, with the blob analysis reliably identifying the platform and extracting its centroid in real time. These centroid coordinates, converted into x and y positional errors, enabled the drone to perform continuous corrective maneuvers while approaching the platform. The **cruise phase** exhibited smooth trajectory adaptation, with the position reference commands (x_{out} , y_{out}) adjusting dynamically based on vision feedback. The proportional control strategy, combined with

velocity scaling, resulted in minimal overshoot and stable alignment. Upon entering the spatial error zone defined for descent, the **altitude control logic** incrementally reduced the drone's height, achieving a **smooth and progressive descent** toward the platform. The **landing sequence** was successfully triggered either mid-motion or at the terminal end of the line, depending on when the drone achieved alignment. The autonomous transition into the Land state, where vertical descent was performed with horizontal lock, resulted in highly accurate touchdowns with negligible drift.

It was observed that the modular structure of the control logic—particularly the clear state separation between hover, cruise, and land—contributed significantly to the robustness and interpretability of the system. The use of vision-driven conditions, such as centroid-based error thresholds, ensured that the descent was only initiated under confident alignment, reducing false positives and improving reliability. However, system performance was notably sensitive to **lighting variability and reflections**. Bright spots or shadows occasionally introduced noise in the HSV segmentation, leading to jitter in the detected centroid. While median filtering mitigated small-scale noise, larger variations could briefly affect tracking stability. This highlights an opportunity for future improvement through adaptive thresholding techniques or more robust color models, such as those used in deep learning-based detection pipelines. Overall, the results validate the efficacy of the approach in achieving real-time, vision-based precision landing on a mobile ground target. The system demonstrated **high reliability, modular design, and successful onboard execution**—all key indicators of readiness for further scaling or application in more complex multi-agent systems.

5. CONCLUSION

This project successfully demonstrated a vision-guided autonomous landing system in which a Parrot Mambo minidrone detects, aligns, and lands on a moving platform carried by a line-follower robot. The system, developed entirely within MATLAB and Simulink, integrates HSV-based image segmentation, blob analysis for centroid tracking, and a structured Stateflow control architecture to achieve real-time onboard decision-making and flight execution. The drone's ability to dynamically interpret visual input, compute positional errors, and initiate a controlled descent highlights the effectiveness of combining lightweight image processing with state-based control logic. The modular design organized around hover, cruise, and land states ensured operational robustness and clarity, while the onboard deployment capability affirmed the computational feasibility of the approach on a resource-constrained embedded platform.

Experimental results confirmed that the drone could reliably detect the colored platform, adjust its trajectory in response to real-time error signals, and land either during the motion of the line-follower or at its terminal point with precision. Despite environmental sensitivity to lighting variations, the system remained operationally stable in most test scenarios. Looking forward, future iterations may incorporate adaptive thresholding or machine learning-based visual classifiers to improve detection reliability under dynamic lighting and textured backgrounds. Additional enhancements may also include inter-agent communication for coordinated timing, or hybrid sensor fusion (e.g., combining vision with ultrasonic or optical flow) to further improve landing robustness. In summary, this work presents a scalable and replicable framework for vision-based drone navigation and mobile platform landing, laying the foundation for broader research in collaborative aerial-ground robotic systems.

6. REFERENCES

1. MathWorks. (n.d.). *Install Windows Bluetooth Drivers*. Retrieved from https://www.mathworks.com/help/simulink/supportpkg/parrot_ug/install-windows-bluetooth-drivers.html
2. MathWorks. (n.d.). *Getting Started with Simulink Support Package for Parrot Minidrones*. Retrieved from <https://www.mathworks.com/help/supportpkg/parrot/ref/getting-started-with-simulink-support-package-for-parrot-minidrones.html>
3. MathWorks. (n.d.). *External Mode for Parrot Minidrones*. Retrieved from <https://www.mathworks.com/help/supportpkg/parrot/ref/external-mode-for-parrot-minidrones.html>
4. MathWorks. (n.d.). *Communicating with Parrot Minidrone Using TCP/IP and UDP*. Retrieved from <https://www.mathworks.com/help/supportpkg/parrot/ref/communicating-with-parrot-minidrone-using-tcpip-and-udp.html>

7. BIOGRAPHIES OF AUTHOR



Chinmay Amrutkar is a master's student at Arizona State University pursuing Robotics and Autonomous Systems (AI). He has completed his Bachelors of Technology in Robotics and Automation at MIT World Peace University, Pune, India. He can be contacted via email at camrutka@asu.edu
